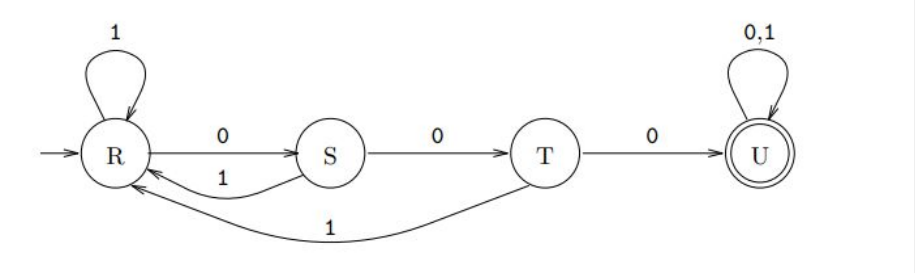
Programski prevodioci

05 Formalni jezici i automati

25-26

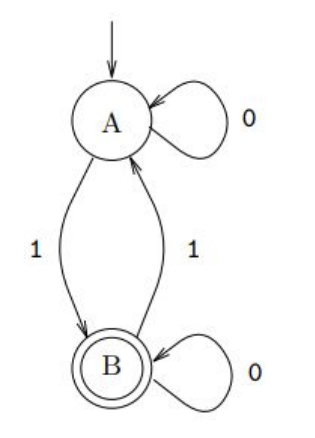
Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad



Koji jezik prihvata ovaj automat?

Reči imaju podstring 000.



Koji jezik prihvata ovaj automat?

Reči imaju neparan broj simbola 1.

Da li od ovih automata možemo da napravimo automat koji proverava

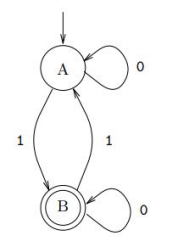
- da li ima neparan broj 1 i ima podniz 000
- da li ima neparan broj 1 ili ima podniz 000
- da li ima neparan broj 1 i nema podniz 000
- da li ima paran broj 1 i ima podniz 000

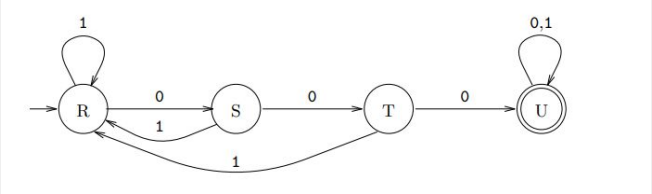
Da li imamo sistematičan način da kombinujemo automate za bilo koje jezike?

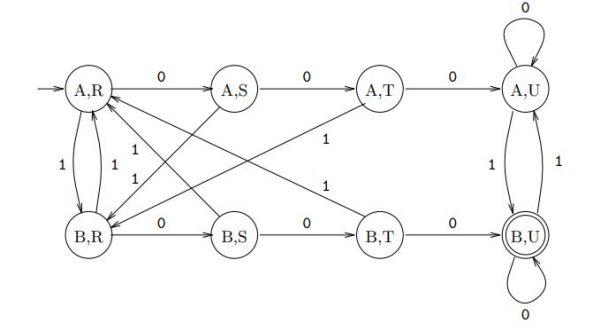
Iz naše perspektive, zašto bi nam to bilo korisno?

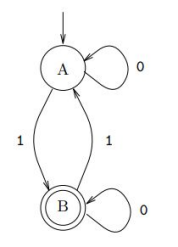
Ne možemo nadovezati, ulaz se čita simbol po simbol

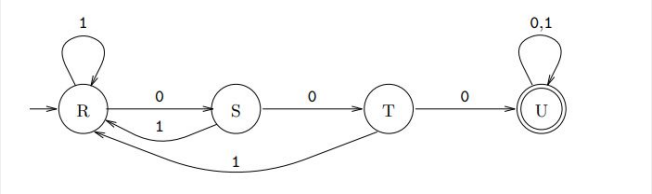
Da li možemo nekako da ih pustimo paralelno da se izvršavaju?
Da se nekako pamte stanja oba automata?

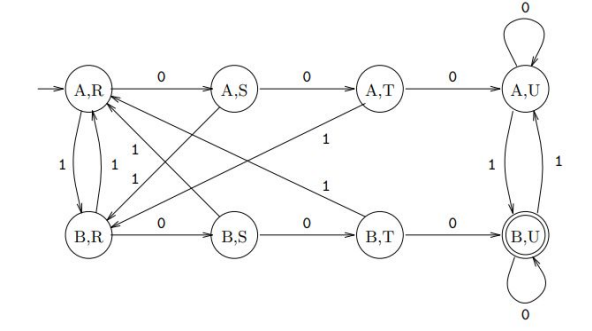

A
B
0
1
1
0

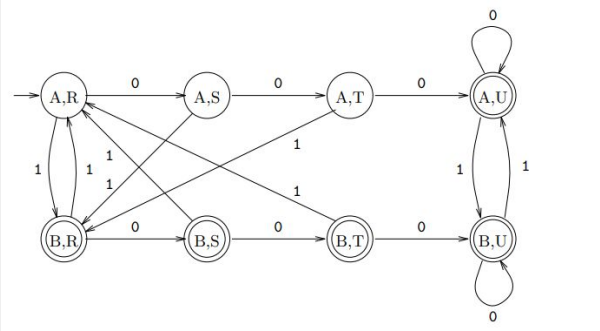

R
S
T
U
1
0
1
0
0
0,1
1


A,R
A,S
A,T
A,U
B,R
B,S
B,T
B,U
0
1









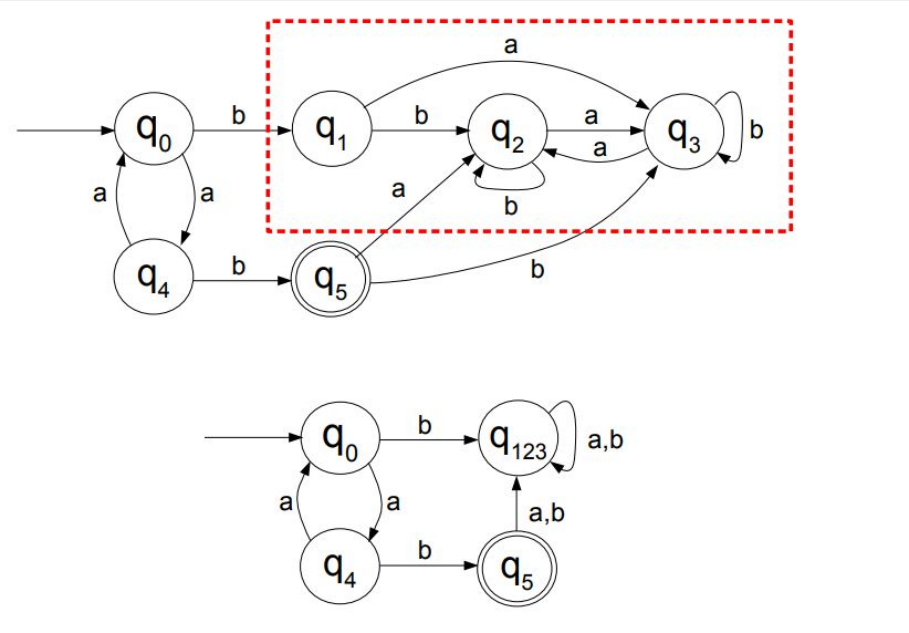
Šta rade novi automati, u čemu je razlika?

Da li imamo sistematičan način da pojednostavimo automat?

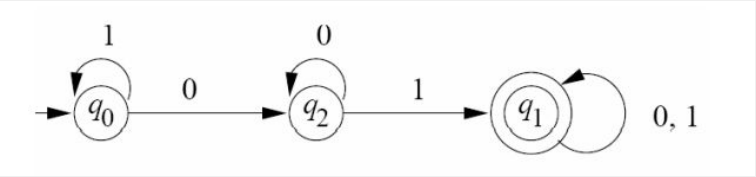
- ekvivalentan: L(A1) = L(A2)
- jednostavniji: manje stanja

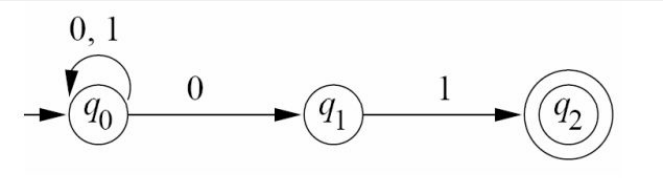
Iz naše perspektive, zašto bi nam to bilo korisno?

Minimizacija: potraga za suvišnim stanjima (nedostižna i ekvivalentna)

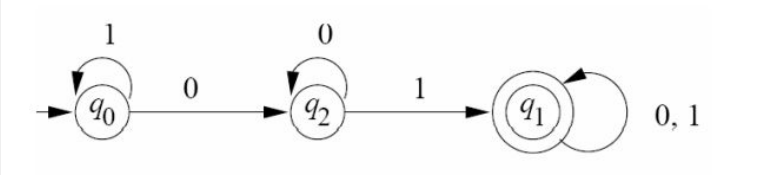


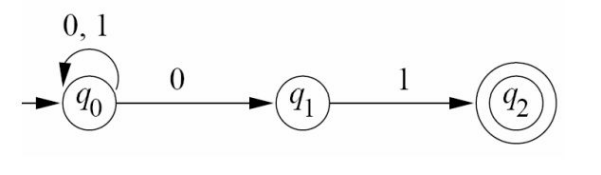
Posmatrajmo sledeće automate:





U čemu je “problem” sa drugim automatom?





Automat je deterministički

Jednoznačno je određena promena stanja za svaki ulazni simbol.

σ, funkcija prelaza, kod DKA uvek vraća jedno stanje.

σ: S x Σ → S

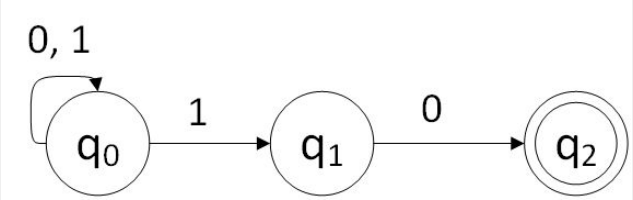
Automat je nedeterministički

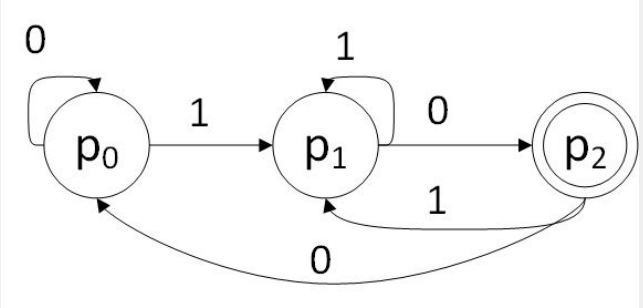
Iz stanja q_0 se na osnovu simbola 0 može preći i u stanje q_0 i u stanje q_1.

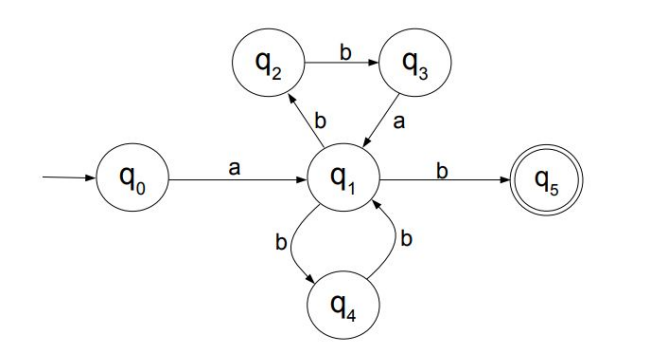
σ, funkcija prelaza, kod NKA vraća podskup stanja.

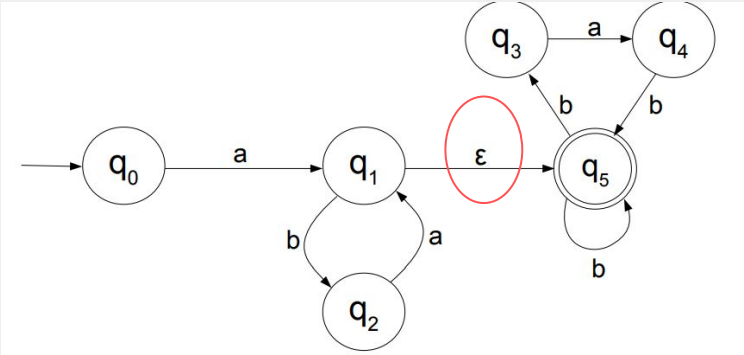
σ: S x Σ → $\mathscr{P}$(S)

- Tvrđenje: Za svaki NKA postoji odgovarajući DKA.
- Prihvataju iste jezike.
- Postoje algoritmi za pretvaranje.
- Zašto ih razmatramo?


0, 1
1
0
q0
q1
q2


0
1
1
0
1
0
p0
p1
p2





a(bba|bb)*b

a(ba)*(b|bab)*

ε-NKA

Postoje prelazi za ε.
Izvršavaju se i bez ulaznih simbola.

DKA

$A = (S, \Sigma, \sigma, s_0, F)$

- S - skup stanja
- Σ - alfabet
- σ - funkcija prelaza, σ: $S \times \Sigma \rightarrow S$
- s_0 - inicijalno stanje
- F - skup ciljnih stanja

NKA

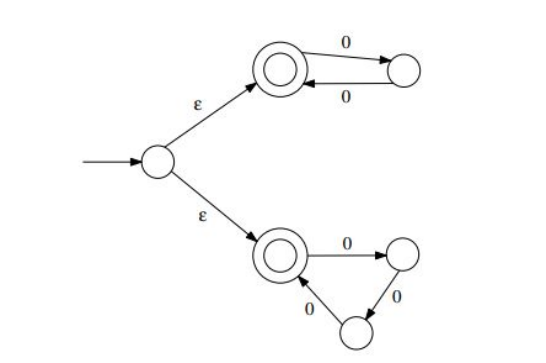
$A = (S, \Sigma, \sigma, s_0, F)$

- S - skup stanja
- Σ - alfabet
- σ - funkcija prelaza, σ: $S \times \Sigma \rightarrow \mathscr{P}(S)$
- s_0 - inicijalno stanje
- F - skup ciljnih stanja

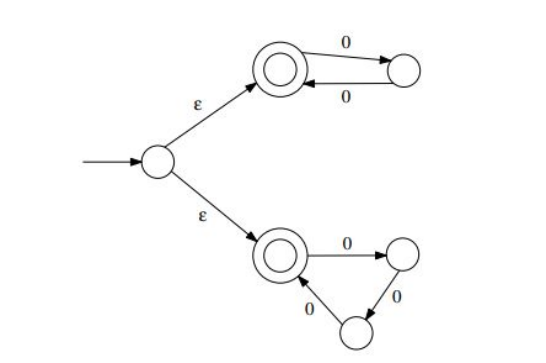
ε - NKA

$A = (S, \Sigma, \sigma, s_0, F)$

- S - skup stanja
- Σ - alfabet
- σ - funkcija prelaza, σ: $S \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathscr{P}(S)$
- s_0 - inicijalno stanje
- F - skup ciljnih stanja



Koji jezik prepoznaje ovaj automat?



Koji jezik prepoznaje ovaj automat?

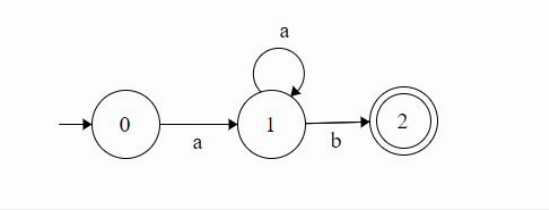
Dužina reči je deljiva sa 2 ili 3

$A = A_1 \cup A_2$

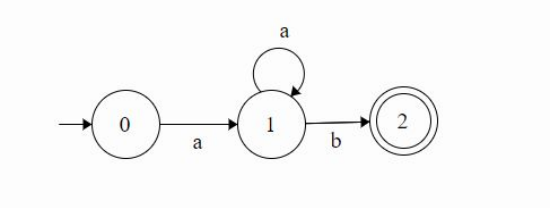
$A_1 = \{0^k : k \equiv 0 \bmod 2\}$
$A_2 = \{0^k : k \equiv 0 \bmod 3\}$

Regularan izraz?

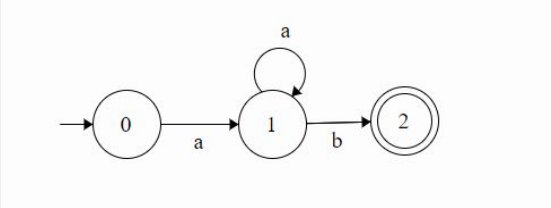
- Tvrđenje: Za svaki e-NKA postoji odgovarajući NKA.
- Prihvataju iste jezike.
- Postoje algoritmi za pretvaranje.
- Zašto ih razmatramo?

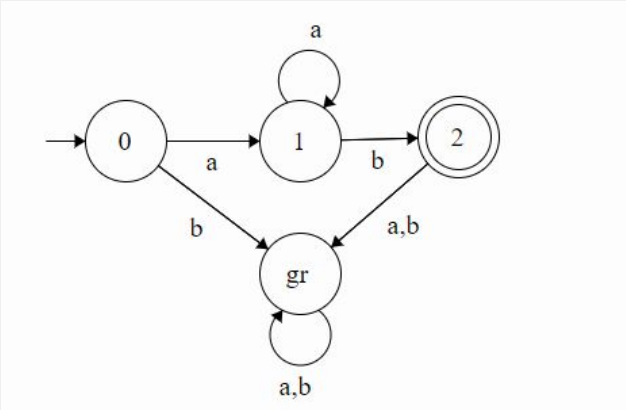


Kako bismo mi implementirali prepoznavanje da li zadata reč pripada jeziku ovog automata?




```
if a
      next
      while a and not eof
            next
      if b
            next
            if EOF
                  OK
            else
                  ERROR
      else
            ERROR
else
      ERROR
```





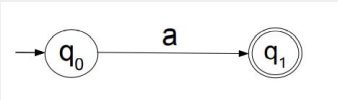

```
if a
      next
      while a  and not eof
            next
      if b
            next
            if EOF
                  OK
            else
                  ERROR
      else
            ERROR
else
      ERROR

if ERROR
      while not eof
            next
```

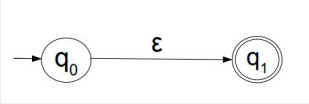
Ali mi nemamo, na primer, automat za leksiku.

Da li postoji neki sistematičan način da dobijemo automat?

- Tvrđenje: Regularni izrazi definišu regularne jezike.
- Tvrđenje: Za svaki regularan jezik postoji konačni automat koji ga prepoznaje.
- Da li imamo sistematičan način da kreiramo automat na osnovu regularnog izraza?
- Iz naše perspektive, zašto bi nam to bilo korisno?

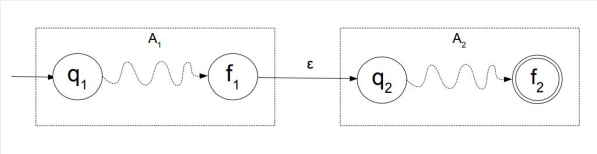


$L(a) = \{a\}$

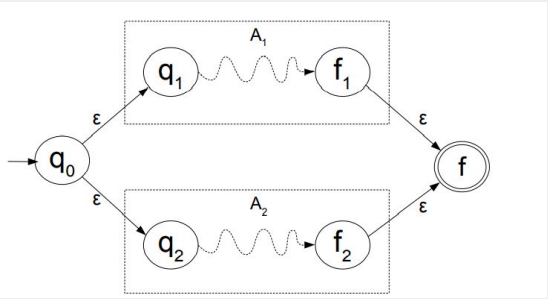


$L(\varepsilon) = \{\varepsilon\}$

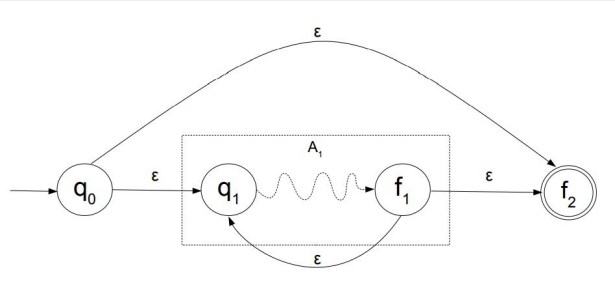
$L(\varnothing) = \varnothing$



$L(r_1 \cdot r_2) = L(r_1)L(r_2)$

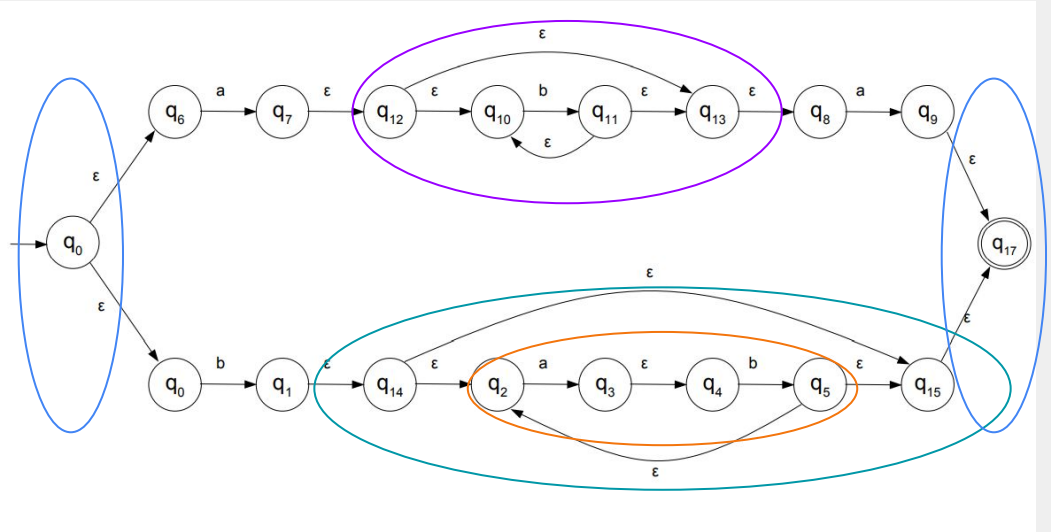


$L(r_1|r_2) = L(r_1) \cup L(r_2)$



$L(r_1^*) = L(r_1)^*$

r = b(ab)* | ab*a

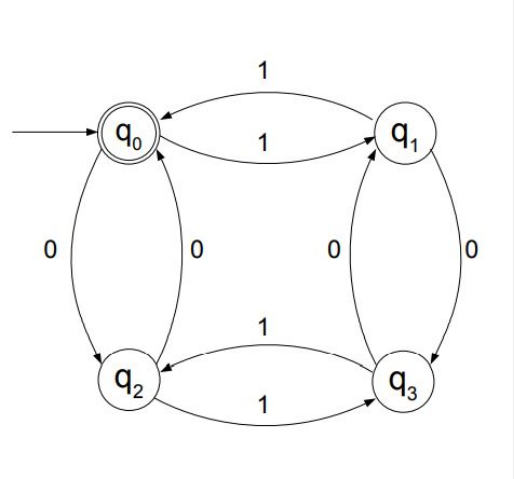


Imamo postupke, algoritme, za:

- kombinovanje
- regularan izraz → eDKA
- eNKA → NKA
- NKA → DKA
- minimizacija

→ kompletna automatizacija procesa

Zadatak



?

može nastati kao proizvod dva slična automata

Zadatak

Konstruisati postupno automat na osnovu regularnog izraza iz primera.

- Regularni jezici su samo jedna podgrupa formalnih jezika.
- Dovoljni su nam za realizaciju leksičke analize pri konstrukciji prevodioca.

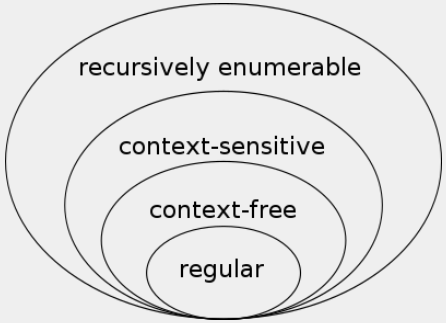
Hijerarhija Čomskog

(Noam Chomsky)

Gramatike:

Tip 3 – regularne gramatike (regularni jezici)
Tip 2 – kontekstno slobodne gramatike (kontekstno slobodni jezici / context free)
Tip 1 – kontekstno osetljive gramatike (kontekstno osetljivi (zavisni) jezici / context sensitive)
Tip 0 – gramatike bez restrikcija (jezici sa fraznom strukturom / rekurzivno nabrojivi)

$T3 \subseteq T2 \subseteq \mathrm{T1} \subseteq T0$ (Tjuringove mašine)

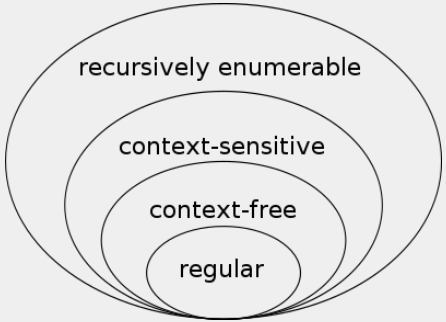


*osnovna podela

Gramatike za opis leksike programskog jezika su znatno jednostavnije nego gramatike za opis sintakse programskog jezika.

Leksika velikog broja programskih jezika predstavlja regularan jezik.

Velik deo sintakse programskih jezika predstavlja kontekstno slobodni jezik.



*osnovna podela

Definicija: Gramatika je uređena četvorka $G = (\Sigma, N, P, S)$ gde je:

- Σ skup terminala (simbola)
- N skup neterminala (pojmova)
- P skup pravila izvođenja
- S početni pojam

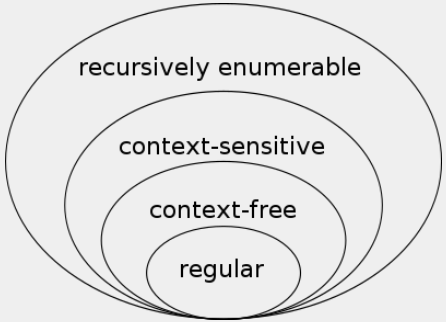
- *Σ, N, P su neprazni, konačni, skupovi*
- $\Sigma \cap N = \varnothing$
- *P je skup pravila u obliku* $(\Sigma \cup N)^* N (\Sigma \cup N)^* \rightarrow (\Sigma \cup N)^*$

regularne: $A \rightarrow a$, $A \rightarrow aB$

kontekstno slobodne: $A \rightarrow \alpha$

kontekstno zavisne: $\alpha A \beta \rightarrow \alpha \gamma \beta$

bez ograničenja: $\alpha \rightarrow \gamma$

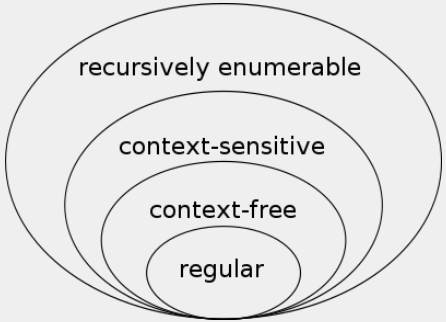


Primer 1

$$S_0 \rightarrow aS_1$$
$$S_1 \rightarrow aS_1$$
$$S_1 \rightarrow bS_1$$
$$S_1 \rightarrow \varepsilon$$

Primer 2

$$S \rightarrow aSa$$
$$S \rightarrow bSb$$
$$S \rightarrow a$$
$$S \rightarrow b$$
$$S \rightarrow \varepsilon$$



$$S \to aSa$$
$$S \to bSb$$
$$S \to a$$
$$S \to b$$
$$S \to \varepsilon$$

Pravila, pre svega i praktično, ukazuju na konkatenaciju, redosled.

Više pravila za isti pojam, praktično, omogućava uniju (alternative).

Često su rekurzivna čime se, praktično, omogućava ponavljanje.

Pravila bez rekurzije, praktično, omogućavaju zaustavljanje rekurzije.

Prazna reč, praktično i između ostalog, omogućava zaustavljanje rekurzije/ponavljanja.

Definicija: Deterministički <u>konačni automat</u> je uređena petorka $A = (S, \Sigma, \sigma, s_0, F)$ gde je:

- S - skup stanja
- Σ - alfabet
- σ - funkcija prelaza, σ: S x Σ → S
- s_0 - inicijalno stanje
- F - skup završnih stanja

Definicija: Deterministički <u>potisni automat</u> je uređena šestorka $A = (S, \Sigma, \Gamma, \sigma, s_0, F)$ gde je:

- S - skup stanja
- Σ - alfabet
- Γ - alfabet steka
- σ - funkcija prelaza, σ: S x (Σ U { _ }) x Γ → S x {N, R} x Γ*
- s_0 - inicijalno stanje
- F - skup završnih stanja

konačni automati - finite automata
potisni automati - push down automata

σ: S x (Σ U { _ }) x $\Gamma \rightarrow$ S x {N, R} x Γ*

σ(s, a, A) = (p, x, w)

automat:

prebacuje se iz stanja s u stanje p

glava trake za čitanje ulaznih simbola:

x = R pomera se za jedan simbol

x = N ne pomera se

stek:

A se menja stringom w (reč nad azbukom Γ)

prihvatanje reči:

- finalno stanje / stek je prazan

Zašto postoji posebna azbuka za stek?

Gramatika i PDA za balansirane zagrade?

Primer konstrukta koji nam je potreban za PL.

Gramatika za balansirane zagrade

$S \rightarrow e \mid SS \mid (S)$

- Drugo pravilo - spoj dva “dela” koji su balansiran i sam je balansiran
- Treće pravilo - balansirani izraz u zagradama je opet balansiran
- prvo pravilo - zaustavi rekurziju

(()(()()()))

ostale zagrade: { }, < >, [ ]?

PDA za balansirane zagrade?

- Kad pročitamo (- stavimo na stek
- Kad pročitamo) - sklonimo sa steka
- Kad obradimo poslednji simbol - stek je prazan

$A = (\Sigma, \Gamma, S, \sigma, s_0)$

- $\Sigma = \{ (,) \}$
- $\Gamma = \{ \$, A\}$
- $S = \{s\}$
- $\sigma: S \times (\Sigma \cup \{ _ \}) \times \Gamma \rightarrow S \times \{N, R\} \times \Gamma^*$
- $s_0 = \{s\}$
- funkcija prelaza:

s($ → sR$A
s(A → sRAA

s)A → sRe
s)$ → sNe - odbacivanje - stek je prazan pre kraja (više desnih)

s_$ → sNe - prihvatanje
s_A → sNA - odbacivanje - string je gotov (više levih)

Obratiti pažnju:

PDA je opis neke mašine.
U opštem slučaju nema veze sa CFG.
Postoje neke dve azbuke i mašina (automat) koji ovako radi.

Slično kao kod regularnih jezika i DFA, ispostavlja se da ovakvi automati prepoznaju CFG.

Tvrđenje: Jezik je kontekstno nezavisan akko postoji PDA koji prihvata taj jezik.

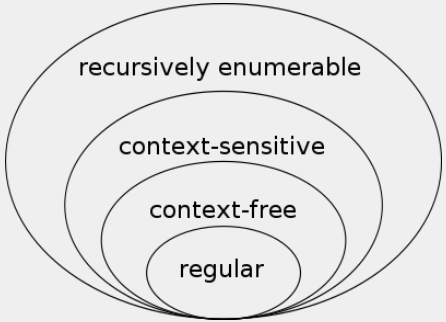
Postoji postupak kako se na osnovu gramatike može generisati potisni automat.

- transformacija gramatike u određeni oblik tako da se jednostavno i direktno dobiju potrebni skupovi

RL: $L = \{a^n b^m \mid n,m >= 1\}$

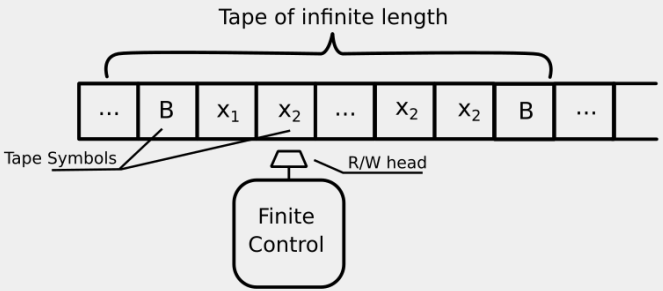
CFL: $L = \{a^n b^n \mid n >= 1\}$, $L = \{a^n b^m c^{n+m} \mid n,m >= 1\}$

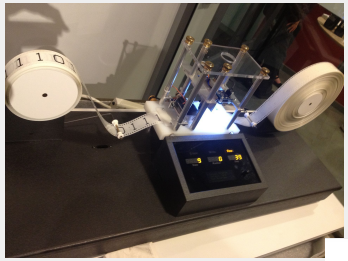
CSL: $L = \{a^n b^n c^n \mid n >= 1\}$, $L = \{a^n b^m c^{nm} \mid n,m >= 1\}$

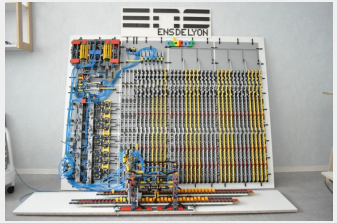


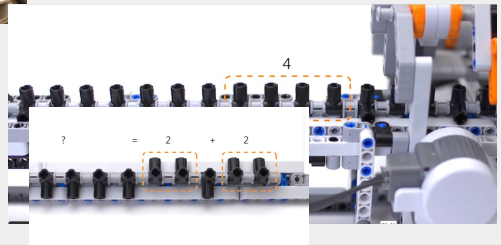
Zadatak

Veći deo sintakse programskih jezika se može definisati konteksto slobodnom gramatiku. Pronaći primere delova ili konstrukta nekih programskih jezika koji su kontestno zavisni.









Wikipedia TM
YouTube TM Minecraft
Vimeo TM Lego

- Tjuringova mašina je najopštija mašina

- linearno ograničen prostor na traci → kontekstno zavisni jezici

- TM sa jednom ili više traka su ekvivalentne

- Univerzalna TM - ulaz i mašina i podaci (računar, arhitektura: instrukcije + podaci)

- Halting problem